PROJECT PROPOSAL

PennyWise : A Price and Ticket Finder for Pakistan

Group Members

1. Areej Raza (SP25-BSE-028)
2. Sannan Ahmad (SP25-BSE-118)

Course: Web Technologies
Instructor: Sir Asif Shehzad

1. Introduction

Buying a product or a ticket in Pakistan usually means opening many tabs. The same phone or pair of shoes is listed at different prices on Daraz, OLX, PriceOye and many smaller stores, and tickets for events, cinemas, buses and flights are spread across separate websites. PennyWise is a web platform that does this searching for the user and shows the cheapest and most reliable options in one place.

2. How the System Works

The user searches for a product or a ticket. The backend runs a web search, takes the top 5 to 10 result pages and crawls them with an open-source crawler (Crawl4AI, *we will test it and choose the final crawler when backend development starts*). The crawled pages are cleaned into structured records containing the title, price, seller, availability and link. Duplicates are removed and the results are ranked by price and relevance. The cleaned data is then sent to a free AI API, which writes a short summary and recommendation. If the AI service is unavailable or its free limit is reached, a fallback module sorts and presents the cleaned results directly, so the site keeps working.

3. Main Features

- **Web product search** with price sorting and filters for tech, clothing and other categories.
- **Product detail** showing all sellers with source links, along with price history.
- **Ticket search** for events and cinema, bus and flights.
- **AI shopping assistant** that explains the best option, with a non-AI fallback.
- **Categories and current deals** for browsing without a search.
- **User accounts** with sign up, sign in, wishlist and price alerts.
- **Admin panel** to manage users, deals, categories, crawl sources and logs.

4. Technology

Frontend: HTML, CSS and Tailwind CSS, with Three.js as an optional visual element. Crawling and AI: Python. The rest of the backend stack will be decided later, based on the technologies we learn in the course; otherwise we will go with FastAPI or Flask. Supabase may be used for authentication and the database once backend development starts.

5. Work Division

The workload is divided evenly between both members, with each contributing 7 to 10 dynamic interfaces. One member leads the backend side, including the crawlers and AI integration, along with the core interfaces of the platform. The other member leads the frontend design of the remaining interfaces, covering the supporting pages and user-facing flows. The detailed split will be finalized as development progresses.

6. Future Scope

Side-by-side product comparison, MongoDB (or database of supabase) for faster search results, scrapers written for specifically our use case in place of generic web-result crawling, and further features and pages as we learn more during development.